

Exercice I : Quel est le danger ?

Soit la section de code suivante exécutée par tous les processus d'un programme MPI (il y'a au moins 3 processus) :

```
int rang, val ; MPI_Status sta ;  
MPI_Comm_rank(MPI_COMM_WORLD, &rang) ;  
if (rang == 0) {  
    MPI_Recv(&val, 1, MPI_INT, MPI_ANY_SOURCE, 1000, MPI_COMM_WORLD, &sta) ;  
} else if (rang == 1) {  
    val = rang ;  
    MPI_Send(&val, 1, MPI_INT, 0, 1111, MPI_COMM_WORLD) ;  
} else if (rang == 2) {  
    val = rang ;  
    MPI_Send(&val, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD) ;  
}
```

- Suite à l'exécution de ces lignes, quelle est la valeur de la variable val sur les 3 premiers processus (justifiez) ?
- Expliquez (brièvement) en quoi cette section de code pose un problème.
- Apportez une correction au problème identifié en b)

Exercice II : Boucles for

Soit le programme OpenMP suivant :

```
void main() {  
    // (A)  
    #pragma omp parallel num_threads(2)  
    for(int i=0 ; i<10 ; i++) {  
        printf("thr=%d, i=%d\n", omp_get_thread_num(), i);  
    }  
  
    // (B)  
    #pragma omp for schedule(static)  
    for(int j=0 ; j<10 ; j++) {  
        printf("thr=%d, j=%d\n", omp_get_thread_num(), j);  
    }  
  
    // (C)  
    #pragma omp parallel for schedule(static) num_threads(2)  
    for(int k=0 ; k<10 ; k++) {  
        printf("thr=%d, k=%d\n", omp_get_thread_num(), k);  
    }  
}
```

Question 1 :

Donnez pour chacune des boucles (A), (B) et (C), le nombre maximal de threads les exécutant. Justifiez brièvement.

Question 2 :

On fait l'hypothèse suivante : chacune des boucles est exécutée par son nombre maximal possible de threads (déterminé en Question 1).

Pour chacune des sections (A), (B) et (C), donnez les intervalles des valeurs de i (respectivement j et k) en fonction des numéros des threads.

Exercice III : Calcul d'une série

Soit S_N une série où N est un entier >0 :

$$S_N = \frac{1}{N} \times \sum_{i=1}^N \frac{u_i}{1+u_i^2}$$

La suite u_i se définit comme suit :

$$\beta = 1.4, u_0 = \frac{-\beta}{2}$$

$$i \geq 1, u_i = u_{i-1} + \beta$$

ce qui peut encore se réécrire :

$$i \geq 1, u_i = u_0 + \beta \times i$$

Soit le programme séquentiel suivant qui calcule dans S la valeur S_N :

```
1 - void main ()
2 - {
3 -   double u, S, beta, sum = 0.0;
4 -   int i, N = 100000;
5 -
6 -   beta = 1.4;
7 -   u = -0.5*beta;
8 -
9 -   for (i=1; i<= N; i++) {
10 -    u += beta;
11 -    sum = sum + u/(1.0+u*u);
12 -  }
13 -
14 -  S = sum/(double)N;
15 -  printf("\n S = %lf\n ", S);
16 - }
```

Travail à faire :

Paralléliser le calcul de S avec OpenMP

Exercice IV : Implémentation d'un gather

Soit la fonction :

```
void allgather( int in, int *out ) ;
```

appelée en même temps par tous les processus MPI.

Elle collecte dans le tableau `out` le contenu de toutes les variables `in` de tous les processus MPI. Tous les

processus doivent récupérer les résultats. Le tableau out est préalablement alloué au nombre total de processus.

La fonction allgather est implémentée dans le listing suivant, ainsi qu'un programme principal qui utilise cette fonction. Cependant le listing n'est pas complet.

Travail à faire : compléter les lignes */* A COMPLETER */* pour que la fonction allgather et le programme principal soient corrects.

Listing de la fonction allgather et son programme principal : complétez-le.

```
1. void allgather( int in, int *out ) {
2.     int rang, P;
3.
4.     /* A COMPLETER : il faut affecter le nombre de processus dans P */
5.     /* A COMPLETER : il faut affecter la variable rang */
6.
7.     if (rang == 0) {
8.         MPI_Status sta;
9.         out[0] = in;
10.        for(int p = 1 ; p < P ; p++) {
11.            int val;
12.            /* A COMPLETER : réception dans val de la valeur envoyée par le processus de rang p */
13.            /* A COMPLETER : mise à jour du tableau out */
14.        }
15.    } else {
16.        /* A COMPLETER : envoi de in au processus 0 */
17.    }
18.    /* A COMPLETER : le processus 0 diffuse out aux autres processus */
19.}
20.
21.int main(int argc, char *argv[])
22.{
23.    int rang, P, *out;
24.
25.    /* A COMPLETER */
26.    /* A COMPLETER : il faut affecter le nombre de processus dans P */
27.    /* A COMPLETER : il faut affecter la variable rang */
28.
29.    out = (int*)malloc(P*sizeof(int));
30.    allgather(rang, out);
31.    for(int p = 0 ; p < P ; p++)
32.        printf("P%d, out[%d] = %d\n", rang, p, out[p]);
33.    free(out);
34.
35.    /* A COMPLETER */
36.    return 0;
37.}
```

Exercice V : Single

Soit le programme OpenMP suivant :

```

#include <stdio.h>
#include <omp.h>

#define N 10

void main() {
    #pragma omp parallel
    {
        int nt = omp_get_num_threads();
        int myrank = omp_get_thread_num();
        for(int k=0 ; k< N ; k++) {
            #pragma omp critical
            {
                printf("critical=%d\n", myrank);
            }

            #pragma omp master
            {
                printf("master=%d\n", myrank);
            }

            #pragma omp single
            {
                printf("single=%d\n", myrank);
            }
        }
    }
}

```

Handwritten notes in the image:
 - Next to `#pragma omp parallel`: 线程 (Thread)
 - Next to `omp_get_thread_num()`: 当前 (Current)
 - Next to the loop `for(int k=0 ; k< N ; k++)`: 10
 - Next to `printf("master=%d\n", myrank);`: 10

Questions 1-*) :

- 1-a) La variable **nt** est-elle locale à chaque thread de la région parallèle ou bien partagée entre ces threads ?
- 1-b) La valeur de **nt** est-elle identique à tous les threads ou bien différente par thread ? Justifiez
- 1-c) Combien va-t-il y avoir d'affichages `printf("critical=%d\n", myrank)` ? Justifiez
- 1-d) Combien va-t-il y avoir d'affichages `printf("master=%d\n", myrank)` ? Justifiez
- 1-e) Combien va-t-il y avoir d'affichages `printf("single=%d\n", myrank)` ? Justifiez
- 1-f) Les valeurs affichées par `printf("master=%d\n", myrank)` sont-elles identiques au cours de l'exécution ? Justifiez
- 1-g) Les valeurs affichées par `printf("single=%d\n", myrank)` sont-elles identiques au cours de l'exécution ? Justifiez

Question 2 : Quelle est l'affirmation correcte ?

- [A] il y a une barrière avant l'accolade ouvrante de "omp single"
- [B] il y a une barrière après l'accolade fermante de "omp single" ✓
- [C] il y a une barrière avant l'accolade ouvrante ET une barrière après l'accolade fermante de "omp single"
- [D] il n'y a aucune barrière avec la construction "omp single"

Question 3 :

On fait l'hypothèse que les threads OpenMP de la région parallèle sont compatibles POSIX. Par conséquent, il est possible de faire appel à la bibliothèque pthread dans la région parallèle

OpenMP.

On désire remplacer la construction OpenMP **single** par des appels pthread.

Travail à faire : Complétez les sections de codes **A**, **B**, **C** et **D** du programme suivant pour reproduire le même comportement que **single** dans le programme d'origine. Indication: vous pouvez utiliser des variables globales.

```
#include <stdio.h>
#include <omp.h>
/* DEBUT SECTION DE CODE A */
includes à compléter
/* FIN SECTION DE CODE A */

/* DEBUT SECTION DE CODE B */
déclarations à compléter
/* FIN SECTION DE CODE B */

#define N 10

int main(int argc, char **argv) {

    #pragma omp parallel
    {
        int nt = omp_get_num_threads();
        int myrank = omp_get_thread_num();
        for(int k=0 ; k< N ; k++) {

            #pragma omp critical
            {
                printf("critical=%d\n", myrank);
            }

            #pragma omp master
            {
                printf("master=%d\n", myrank);
            }

            /* DEBUT SECTION DE CODE C */
            à compléter
            /* FIN SECTION DE CODE C */
            {
                printf("single=%d\n", myrank);
            }

            /* DEBUT SECTION DE CODE D */
            à compléter
            /* FIN SECTION DE CODE D */
        }
    }
    return 0;
}
```

Exercice VI : Recouvrement

Dans la section de code MPI suivante, la fonction **collect_trait** est utilisée dans les conditions suivantes :

1. cette fonction est appelée par tous les processus MPI (comme une opération collective) ;
2. chaque processus a pré-alloué et a rempli son propre tableau tab de N entiers ;

3. la valeur de N est identique pour tous les processus MPI ;
4. la fonction `trait(int *tab, int N)` (appelée par le processus 0) effectue un traitement coûteux à partir des données du tableau `tab` passé en argument, sans modifier le contenu de ses données.

天 增加 7. 处理

```
void collect_trait(int *tab, int N)
{
    int rank, P, k;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    if (rank == 0)
    {
        MPI_Status sta;
        int *buf2 = (int*)malloc(N*sizeof(int));

        trait(tab, N);

        for(k = 1 ; k < P ; k++)
        {
            MPI_Recv(buf2, N, MPI_INT, k, 1000, MPI_COMM_WORLD, &sta);
            trait(buf2, N);
        }

        free(buf2);
    }
    else
    {
        MPI_Send(tab, N, MPI_INT, 0, 1000, MPI_COMM_WORLD);
    }
}
```

□ □ □ □

Question 1 : quelle est la modification à apporter à la réception pour que le processus 0 reçoive les messages dans n'importe quel ordre ?

带来

Question 2 : expliquez brièvement pourquoi la réception (par le processus 0) n'est pas recouverte par le traitement `trait`.

使用

Question 3 : modifiez la section de code exécutée par le processus 0 pour recouvrir la réception par le traitement `trait` tout en recevant les messages dans n'importe quel ordre.

Indication : utilisez deux buffers de N entiers, un pour recevoir des données, l'autre pour traiter des données déjà reçues.

同时

